



Universidade Federal de Viçosa – Campus
UFV-Florestal
Ciência da Computação – Engenharia de Software II
Projeto Integrador 2025

Modelo de Classes - sprint 00

Designer de Sistemas

Equipe 03 - Jogo Desconecta

Ana Lúcia de Souza - 5084

Florestal

2025

Sumário

Back-end.....	3
Classes de Entidade (Entity):.....	4
Classe de Controle:.....	5
Classe de acesso ao Banco de Dados:.....	5
Relacionamento entre as classes:.....	6
Front-end.....	7

Back-end

A figura 1 mostra o Diagrama de Classes completo para os Casos de Uso da Sprint 01, isto é, **CSU02: Realizar Cadastro** e **CSU03: Autenticar aluno (Login)**. Observe que esse diagrama utiliza a categorização BCE (Boundary, Control, Entity), a qual implica que cada objeto é especialista em realizar um dos três tipos de tarefa, a saber: comunicar-se com atores (fronteira), manter as informações (entidade) ou coordenar a realização de um caso de uso (controle).

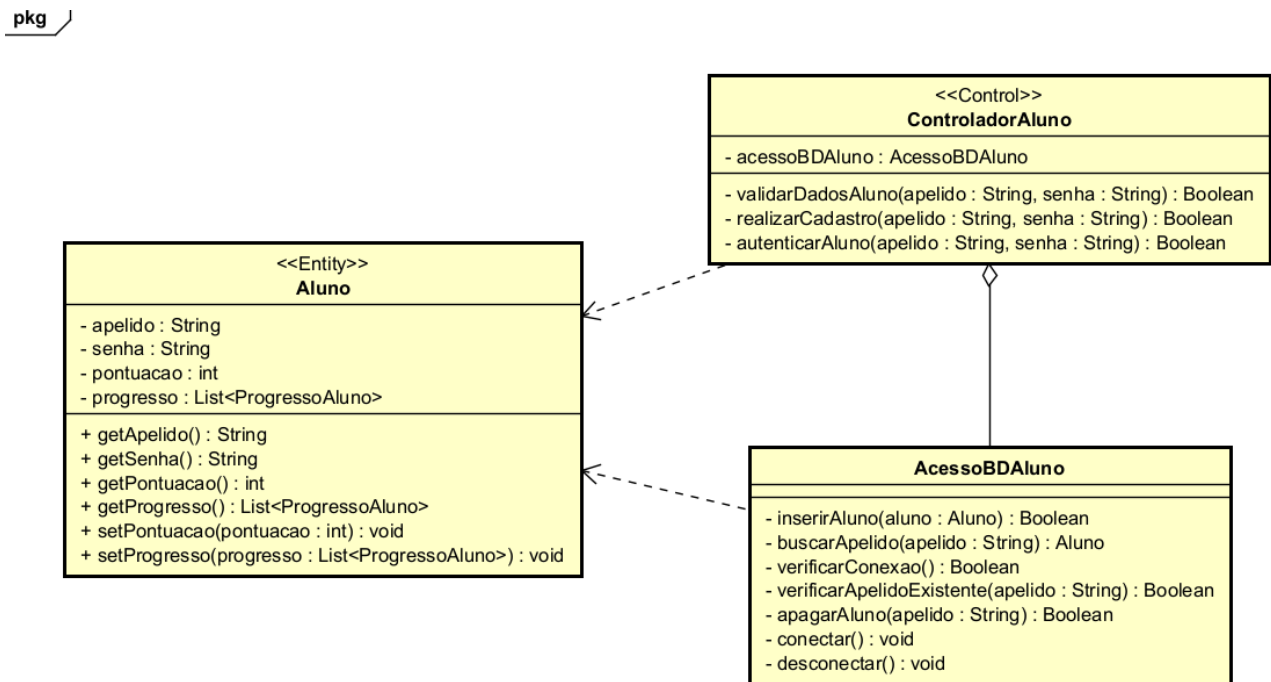


Figura 1. Diagrama de Clases para CSU02 e CSU03

A figura 2 ilustra de forma esquemática a interação entre os objetos de um caso de uso: o ator se comunica com um objeto de fronteira, que repassa (pelo envio de mensagens) as ações desse ator para o objeto de controle (controlador). O controlador, por sua vez, coordena a colaboração de um ou mais objetos da entidade para a realização da tarefa desejada pelo ator. Note que, de acordo com o esquema dessa Figura, os objetos de entidade também se comunicam entre si com o envio de mensagem. Note também que a realização de uma tarefa pode envolver o envio de mensagens para outros objetos de fronteira, para que o sistema se comunique com outros atores (por exemplo, outros sistemas) envolvidos no caso de uso em questão. Após a conclusão da tarefa, o objeto de controle repassa o resultado para um objeto de fronteira (que pode ser o mesmo no qual a realização começou, ou outro objeto). Este objeto de fronteira, por sua vez, apresenta o resultado para o ator.

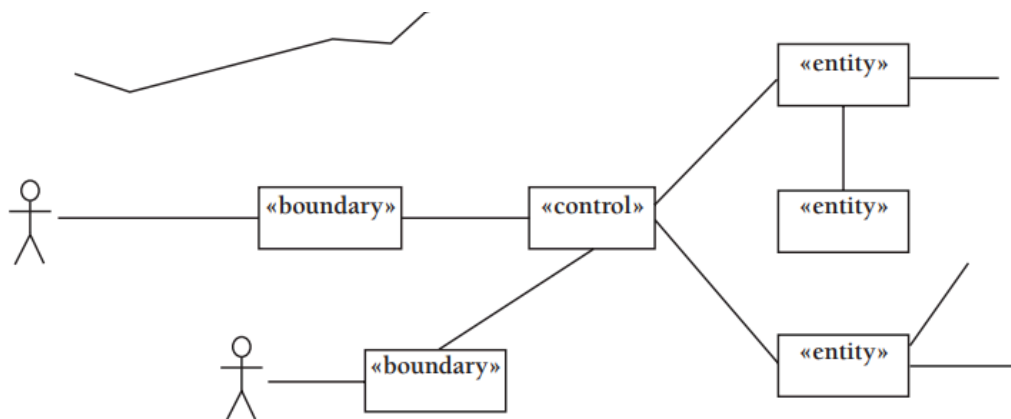


Figura 2. Representação de uma interação BCD

Observe, ainda, que não há no diagrama de classes da Figura 1 a representação da classe de fronteira e isto se dá pela razão de este projeto utilizar em seu front-end o React, uma biblioteca que não segue o paradigma orientado a objetos. Sendo assim, por mais que o UML seja extensível, não é padrão utilizar o diagrama de classes para o designer de sistemas não orientado a objetos. Por esse motivo, seguiremos com outra abordagem para este caso especial, ou seja, a diagramação das classes que seriam de fronteira, ou seja, as telas as quais compõem o front-end deste projeto. Tal abordagem será detalhada no tópico “front-end”.

Classes de Entidade (Entity):

Para os dois casos de uso presentes nesta sprint, apenas a Classe Aluno é necessária como classe de entidade. Sendo assim, apenas esta classe deverá ser utilizada como base para a construção de tabelas pelos alunos da disciplina Banco de Dados neste momento.

Nesta classe, há quatro atributos privados, sendo eles: apelido (nome do aluno), senha, pontuacao (número de pontos conquistado pelo aluno durante os desafios), e progresso (estado que o aluno parou de jogar, ou seja, qual desafio foi realizado por último pelo aluno).

O atributo progresso merece uma atenção especial: observe que ele utiliza uma lista do tipo “ProgressoAluno”, porém esta classe não faz parte dos casos de uso dessa sprint e, portanto, deve ser modelada em outro momento. Sendo assim, o programador deve apenas declarar esta classe sem se preocupar em implementá-la neste momento.

Nesta classe, também há quatro métodos getters e apenas dois métodos setters. Essa escolha foi feita pelo motivo de que o jogador não poderá alterar seu apelido e senha mesmo em caso de esquecimento dos mesmos.

Classe de Controle:

A classe “ControladorAluno”, possui o atributo privado `acessoBDAluno`, que será criado pelo construtor dessa classe.

Nessa classe também há os seguintes métodos:

- `validarDadosAluno(apelido: String, senha: String): Boolean`: Esse método tem como objetivo verificar se todos os campos (apelido e senha) foram preenchidos e possuem formato válido.
- `realizarCadastro(apelido: String, senha: String): Boolean`: Esse método tem o objetivo de criar um cadastro para o aluno, em outras palavras, deve-se criar uma instância de `Aluno` com os dados que foram fornecidos pelo usuário e corretamente validados pelo método “`validarDados()`”. Ademais, o método também deve verificar se o apelido escolhido pelo usuário é único. Para isso, a função “`verificarApelidoExistente()`” da Classe `AcessoBDAluno` deve ser utilizada. Além disso, esse aluno criado deve ser salvo no banco de dados através do método “`inserirAluno()`” da classe “`AcessoBDAluno`” e verificar se a inserção desses dados no banco foi bem sucedida.
- `autenticarAluno(apelido: String, senha: String): Boolean`: Esse método tbm deve utilizar o método “`validarDados()`” a fim de verificar se os campos de login (apelido e senha) foram preenchidos. Além disso, deve-se chamar o método “`buscarApelido()`” da classe `AcessoBDAluno` com o intuito de verificar se há no banco de dados o apelido cadastrado no qual o usuário tenta fazer login. Se “`buscarApelido()`” não for nulo, ou seja, se um `Aluno` foi retornado por esse método, significa que há no banco um usuário com esse apelido cadastrado. Então, deve-se comparar a senha fornecida durante o processo de login com a senha armazenada no objeto `Aluno` retornado por “`buscarApelido`”. Caso ambas as senhas sejam idênticas, a autenticação foi bem sucedida.

Classe de acesso ao Banco de Dados:

A classe `AcessoBDAluno` será a responsável pelo acesso e manipulação do banco de dados. Ela possui o atributo privado “`conexao`”, e seus métodos serão descritos a seguir:

- `inserirAluno(aluno: Aluno): Boolean`: Esse método recebe um `Aluno` e o insere na tabela previamente criada para `Aluno` no banco de dados.

- `verificarApelidoExistente(apelido: String): Boolean`: Esse método consulta o banco de dados para verificar se o apelido o qual o usuário tenta cadastrar já existe no banco de dados.
- `buscarApelido(apelido: String) : Aluno`: Esse método também consulta o banco de dados com o intuito de verificar se o apelido já existe nesse banco, porém, ao contrário do método anterior, retorna um objeto `Aluno` preenchido com os dados presentes nessa linha da tabela ou nulo se o apelido não for encontrado.
- `apagarAluno(apelido: String): Boolean`: Esse método pode ser usado pelos desenvolvedores para apagar os alunos que forem inseridos no banco com o propósito de testar o funcionamento dos métodos e do sistema.

Relacionamento entre as classes:

Observe que entre as classes `Aluno` e `ControladorAluno` e entre as classes `Aluno` e `AcessoBDAluno` há um relacionamento de dependência.

A dependência é um relacionamento no qual a mudança de uma classe pode afetar o comportamento ou estado de outra classe. As dependências podem ser usadas no contexto de classes, para mostrar que uma classe usa outra como um argumento na assinatura de uma operação ou uma variável local. Observe neste momento, apenas a relação de dependência entre `ControladorAluno` e `AcessoBDAluno`: o método `inserirAluno` recebe em seu parâmetro um objeto da Classe `aluno`, além disso, o método `buscarApelido` retorna um objeto da mesma classe, o que configura uma relação de dependência entre as duas classes, em outras palavras, `AcessoBDAluno` depende de `Aluno`.

Observe agora a relação entre as classes `Aluno` e `ControladorAluno`: não há nos métodos de `ControladorAluno` um parâmetro ou um valor de retorno que seja um objeto `Aluno`. Então por que ainda assim, existe a relação de dependência entre as classes? A dependência aqui é uma dependência por variável - que é mais difícil ser identificada ao se olhar apenas para o diagrama UML. Observe, por exemplo, que o método `realizarCadastro` precisa criar uma instância de `Aluno`, assim, surge uma relação de dependência entre as classes por causa desse objeto (variável) criado. Esse objeto será usado localmente dentro do método, ou seja, quando o método termina, o objeto não existirá mais. Observe ainda, que a Classe `ControladorAluno` não possui um atributo do tipo `Aluno`, sendo assim, aqui a relação não poderia ser de agregação composta, que também instancia objetos de outra classe.

Por fim, observe a relação entre ControladorAluno e AcessoBDAluno: essa é uma relação de agregação simples. Nesse tipo de relacionamento, a destruição de um objeto todo não implica na destruição do objeto parte. Aqui o objeto todo é ControladorAluno e AcessoBDAluno é um objeto parte. Se destruímos ControladorAluno, AcessoBDAluno continua existindo.

Front-end

Para a modelagem do front-end, foi utilizado o diagrama de máquina de estados conforme apresentado na figura 3.

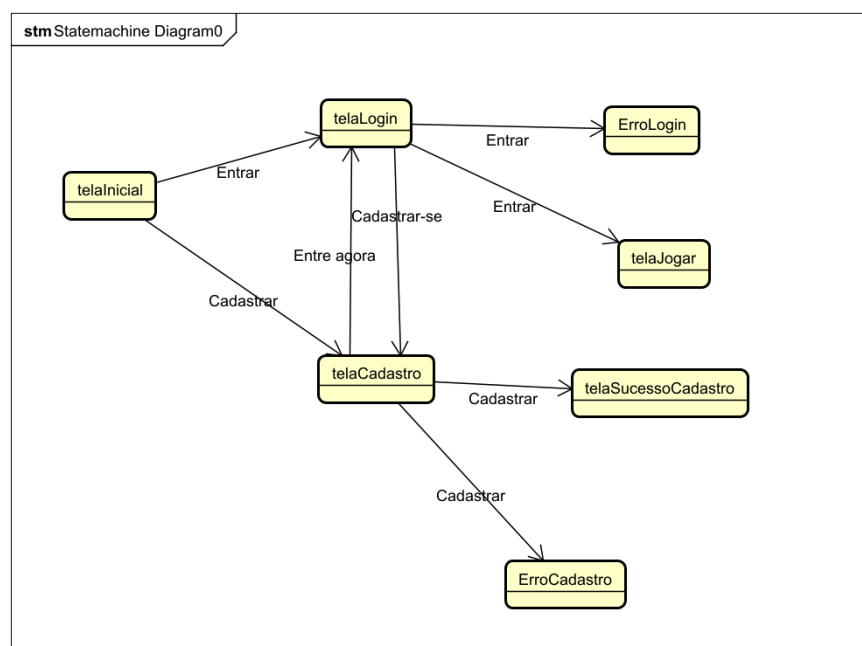


Figura 3. Modelagem front-end.

A figura 4 representa a “telaInicial” do programa. Ao clicar no botão Entrar, o aluno é direcionado para o estado “telaLogin” (Figura 5). Ao clicar em Cadastrar, o aluno é direcionado para o estado “telaCadastro” (Figura 6).



Figura 4. Prototipagem de telaInicial.



Figura 5. Prototipagem de telaLogin.



Figura 6. Prototipagem de telaCadastro.

Estando no estado “telaCadastro” (Figura 5), ao clicar em “Entre agora”, o aluno é direcionado para o estado da Figura 4. Já se o aluno se encontrar no estado representado pela figura 4, ao clicar no botão “Cadastre-se”, ele é direcionado para o estado representado pela Figura 5.

A partir do estado representado pela Figura 5, dois estados são possíveis ao se clicar no botão “Cadastrar”: “telaSucessoCadastro” (Figura 7) e tela “erroCadastro” (Figura 8).



Figura 7. Prototipagem de telaSucessoCadastro.

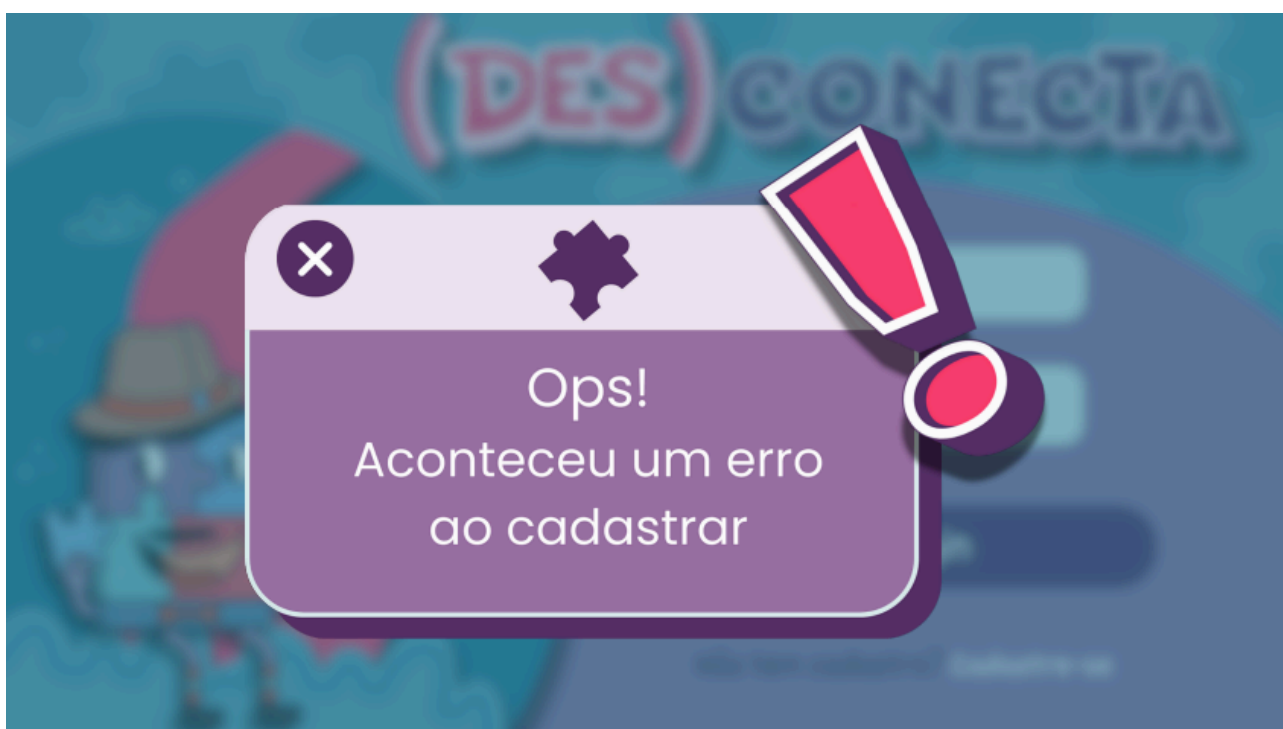


Figura 8. Prototipagem de ErroCadastro.

A partir do estado representado pela Figura 4, dois estados são possíveis ao se clicar no botão “Entrar”: “telaJogar” (Figura 9) e tela “erroLogin” (Figura 10).



Figura 9. Prototipagem de telaJogar.

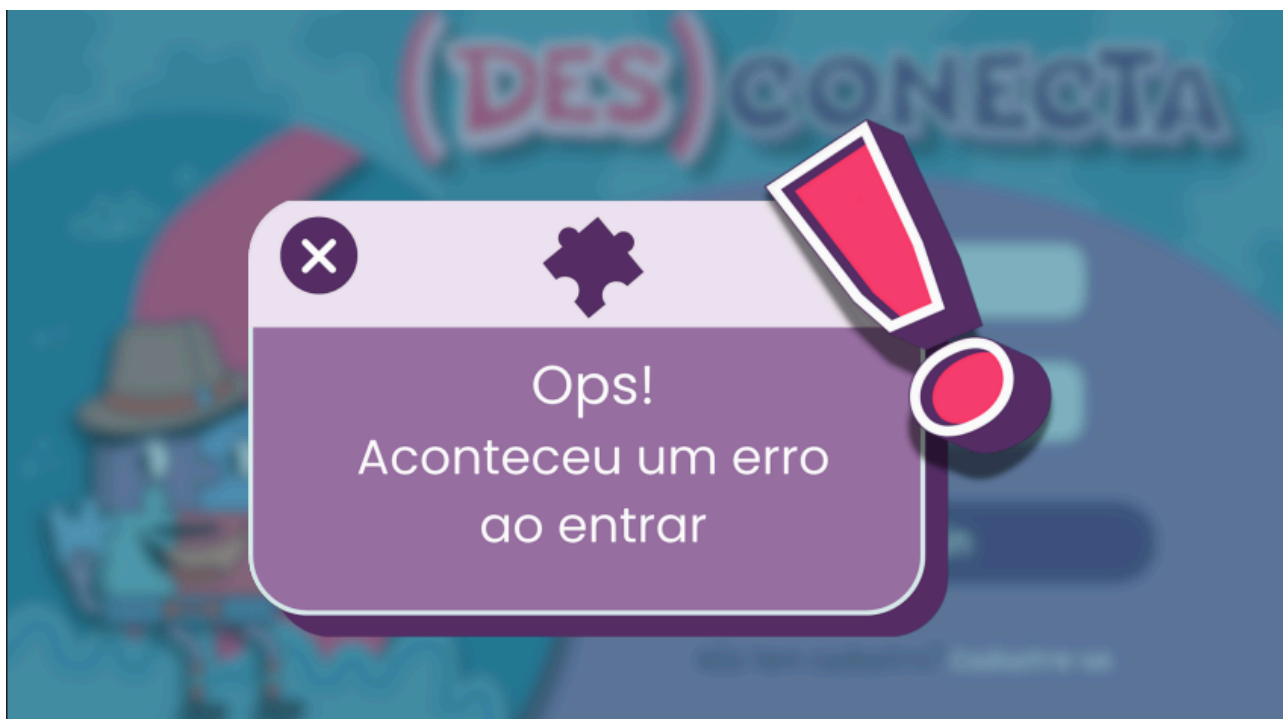


Figura 10. Prototipagem de erroLogin.